

遥感目标检测平台完整环境安装配置教程

技术栈：Vue3 + Element Plus + FastAPI + Ultralytics YOLO11 + SQLite

适用系统：Windows 10 21H2+/Windows 11 22H2+（推荐）、Ubuntu 20.04/22.04 LTS、macOS 12+

版本锁定原则：所有依赖均采用长期稳定兼容版本，禁止使用未验证的最新版，避免兼容性问题

最终验证标准：前后端服务正常启动、YOLO11 GPU 推理可用、数据库读写正常、Git 代码托管正常

一、前置硬件与系统要求

1.1 最低硬件配置

组件	要求	说明
CPU	Intel i5-10400F / AMD R5 3600 及以上	负责数据预处理和后端服务调度
GPU	NVIDIA GTX 1660 Ti (显存 ≥6GB)	训练 / 推理必须，显存 8GB 以上最佳；AMD/Apple Silicon 仅支持 CPU 推理
内存	16GB DDR4 及以上	避免 YOLO 训练时内存溢出
存储	50GB 可用 SSD 空间	存放数据集、模型权重和项目代码

1.2 系统与驱动要求

系统	最低版本	特殊说明
Windows	10 21H2 / 11 22H2	需安装 NVIDIA 显卡驱动≥535.104.05 (对应 CUDA 12.1)
Ubuntu	20.04 LTS / 22.04 LTS	需安装 NVIDIA 驱动和 CUDA Toolkit
macOS	12 Monterey+	仅支持 CPU 推理，不推荐用于模型训练

二、基础工具安装（跨平台通用）

2.1 Git 安装与配置

作用：代码版本控制与 GitHub 托管

安装步骤

- 下载对应系统安装包：[Git 官方下载](#)
- 安装时保持默认选项，Windows 勾选 "Add Git to PATH"
- 验证安装：打开终端 / 命令提示符，执行

```
git --version
# 输出示例: git version 2.45.1.windows.1
```

全局配置

```
# 配置用户名 (与GitHub账号一致)
git config --global user.name "你的GitHub用户名"

# 配置邮箱 (与GitHub注册邮箱一致)
git config --global user.email "你的GitHub邮箱"

# 配置默认分支名
git config --global init.defaultBranch main

# 验证配置
git config --list
```

2.2 Python 安装与虚拟环境

版本要求: 3.10.x 或 3.11.x (禁止 3.12 及以上, Ultralytics YOLO11 不兼容)

安装步骤

1. 下载对应系统安装包: [Python 3.10.15 官方下载](#)
2. Windows 安装时勾选 "Add Python 3.10 to PATH", 选择 "Customize installation"→勾选 "Install for all users"
3. 验证安装:

```
python --version # Windows
python3 --version # Linux/macOS
# 输出示例: Python 3.10.15
```

虚拟环境创建 (推荐 Anaconda)

Anaconda 可统一管理 Python 环境和 CUDA 依赖, 避免版本冲突

1. 下载安装: [Anaconda 官方下载](#)
2. 创建项目专属虚拟环境:

```
# 创建环境 (命名为rsod-web)
conda create -n rsod-web python=3.10.15 -y

# 激活环境
conda activate rsod-web

# 验证Python版本
python --version
```

虚拟环境创建 (venv)

```
# 进入项目根目录（包含backend和frontend的上层目录）
cd rsod-web-platform/backend

# 创建虚拟环境（命名为.venv，行业通用命名）
python3 -m venv .venv

# 执行完成后，项目根目录会生成一个`.venv`文件夹，包含独立的 Python 解释器和 pip。
source .venv/bin/activate
```

✅ 激活成功后，终端提示符前会出现 `(.venv)` 标识：

```
(.venv) lily@LilyMacBook-Pro backend %
```

2.3 Node.js 安装

版本要求：18.x LTS 或 20.x LTS（Vite 构建工具要求）

安装步骤

1. 下载对应系统 LTS 版本：[Node.js 官方下载](#)
2. Windows 安装时保持默认选项，自动添加到 PATH
3. 验证安装：

```
node --version
# 输出示例：v20.15.1

npm --version
# 输出示例：10.7.0
```

配置国内镜像源（加速依赖下载）

```
npm config set registry https://registry.npmmirror.com

# 验证配置
npm config get registry
# 输出：https://registry.npmmirror.com
```

三、IDE 安装与配置

3.1 VS Code（前端开发）

1. 下载安装：[VS Code 官方下载](#)
2. 必装插件：

插件名称	插件 ID	作用
Vue - Official	Vue.volar	Vue3 语法高亮与智能提示
Element Plus Snippets	sdras.vue-vscode-snippets	Element Plus 代码片段
ESLint	dbaeumer.vscode-eslint	代码规范检查
Prettier	esbenp.prettier-vscode	代码格式化
GitLens	eamodio.gitlens	Git 历史可视化

配置保存自动格式化：

- 打开设置（Ctrl+,）→ 搜索 "Format On Save" → 勾选
- 设置默认格式化工具为 Prettier

3.2 PyCharm（后端开发）

1. 下载安装：[PyCharm 社区版 官方下载](#)（免费足够使用）
2. 配置 Python 解释器：
 - 打开项目 → File → Settings → Project: rsod-web → Python Interpreter
 - 点击 "Add Interpreter" → 选择 "Conda Environment" → 选择已创建的 `rsod-web` 环境
3. 必装插件：
 - FastAPI：提供 FastAPI 语法提示和路由跳转
 - Pydantic：提供 Pydantic 模型智能提示
 - Database Tools：内置 SQLite 数据库可视化工具

四、后端环境配置（FastAPI + YOLO11 + SQLite）

4.1 核心依赖安装

新建：`backend/requirements.txt`

```
fastapi==0.104.1
uvicorn==0.24.0
python-multipart==0.0.6
pillow==10.1.0
ultralytics==8.0.200
opencv-python==4.8.1.78
pydantic==2.5.2
sqlalchemy==2.0.30
alembic==1.13.2
```

激活 `rsod-web` 虚拟环境后，执行以下命令：

```
pip install -r requirements.txt
```

4.2 CUDA 可用性验证（关键步骤）

创建 `test_cuda.py` 文件，写入以下代码并运行：

```
import torch
import ultralytics

print(f"PyTorch版本: {torch.__version__}")
print(f"CUDA是否可用: {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"GPU型号: {torch.cuda.get_device_name(0)}")
    print(f"CUDA版本: {torch.version.cuda}")
    print(f"显存大小: {torch.cuda.get_device_properties(0).total_memory / 1024**3:.2f} GB")

print(f"\nUltralytics版本: {ultralytics.__version__}")
# 测试YOLO11推理
model = ultralytics.YOLO("yolov11n.pt")
results = model("https://ultralytics.com/images/bus.jpg")
print("\nYOLO11推理测试成功!")
```

✅ 成功输出：显示 GPU 型号、CUDA 版本和显存大小，且 YOLO 推理无报错

❌ 失败排查：

- 检查 NVIDIA 驱动版本是否≥535.104.05
- 确认安装的 PyTorch 版本与 CUDA 版本匹配
- 重启终端重新激活虚拟环境

五、前端环境配置（Vue3 + Element Plus）

5.1 Vue3 + Vite 项目初始化

```
# 创建Vite+Vue项目
npm create vite@latest frontend -- --template vue

# 进入项目目录
cd frontend

# 安装基础依赖
npm install
```

5.2 核心依赖安装

```
# 1. Element Plus 组件库
npm install element-plus@2.7.6

# 2. 自动导入插件（无需手动import组件）
npm install -D unplugin-vue-components@0.27.0 unplugin-auto-import@0.17.6

# 3. 路由与HTTP请求
npm install vue-router@4.4.0 axios@1.7.2
```

创建路由文件

在 `frontend/src` 里新建文件夹 `router`,

然后新建文件 `index.js`

完整路径:

```
frontend/src/router/index.js
```

把下面代码全部复制进去:

```
// router/index.js
import { createRouter, createWebHistory } from "vue-router";
import index from "../views/index.vue"; // 你的检测页面

// 路由配置
const routes = [
  {
    path: "/",
    name: "index",
    component: index, // 默认打开就是检测页面
  },
];

// 创建路由实例
const router = createRouter({
  history: createWebHistory(),
  routes,
});

export default router;
```

5.3、在 main.js 中挂载路由

打开你的 `frontend/src/main.js`

替换成下面完整代码:

```
import { createApp } from 'vue'
import App from './App.vue'
import router from './router/index.js' // 引入路由

const app = createApp(App)

app.use(router) // 启用路由
app.mount('#app')
```

5.4、修改 App.vue（路由出口）

全部替换成：

```
<template>
  <!-- 路由出口 → 页面会在这里显示 -->
  <router-view />
</template>

<script setup>
</script>

<style>
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

#app {
  font-family: Avenir, Helvetica, Arial, sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  text-align: center;
  color: #2c3e50;
  margin-top: 30px;
}
</style>
```

5.5 创建测试页面

创建纯手动实现的滑块对比组件

```
<template>
  <div class="slider-compare" ref="sliderRef">
    <!-- 左侧原图 -->
    <div class="image-layer image-before">
      
    </div>

    <!-- 右侧标注图（宽度动态变化） -->
```

```

<div class="image-layer image-after" :style="{ width: sliderPosition + '%' }">
  
</div>

<!-- 滑块手柄 -->
<div
  class="slider-handle"
  :style="{ left: sliderPosition + '%' }"
  @mousedown="startDrag"
  @touchstart="startDrag"
>
  <div class="handle-icon">↔</div>
</div>
</div>
</template>

<script setup>
import { ref, onUnmounted } from 'vue'

const props = defineProps({
  before: {
    type: String,
    required: true
  },
  after: {
    type: String,
    required: true
  }
})

const sliderRef = ref(null)
const sliderPosition = ref(50) // 默认中间位置
let isDragging = false

const startDrag = () => {
  isDragging = true
  document.addEventListener('mousemove', onDrag)
  document.addEventListener('mouseup', stopDrag)
  document.addEventListener('touchmove', onDrag)
  document.addEventListener('touchend', stopDrag)
}

const onDrag = (e) => {
  if (!isDragging || !sliderRef.value) return
  e.preventDefault()

  const rect = sliderRef.value.getBoundingClientRect()
  const clientX = e.touches ? e.touches[0].clientX : e.clientX
  const x = clientX - rect.left
  const width = rect.width

  // 限制滑块在0-100%范围内

```



```

    let position = (x / width) * 100
    position = Math.max(0, Math.min(100, position))
    sliderPosition.value = position
  }

  const stopDrag = () => {
    isDragging = false
    document.removeEventListener('mousemove', onDrag)
    document.removeEventListener('mouseup', stopDrag)
    document.removeEventListener('touchmove', onDrag)
    document.removeEventListener('touchend', stopDrag)
  }

  onUnmounted(() => {
    stopDrag()
  })
</script>

<style scoped>
.slider-compare {
  position: relative;
  width: 100%;
  height: 500px;
  overflow: hidden;
  border-radius: 8px;
  user-select: none;
  cursor: ew-resize;
}

.image-layer {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
}

.image-layer img {
  width: 100%;
  height: 100%;
  object-fit: contain;
}

.image-after {
  overflow: hidden;
  border-right: 3px solid #fff;
}

.slider-handle {
  position: absolute;
  top: 0;
  bottom: 0;

```

```

width: 3px;
background-color: #fff;
z-index: 10;
}

.handle-icon {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  width: 40px;
  height: 40px;
  border-radius: 50%;
  background-color: #fff;
  box-shadow: 0 2px 8px rgba(0,0,0,0.3);
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 18px;
  font-weight: bold;
  color: #333;
}
</style>

```

在检测页面使用组件

打开你的智能检测页面（如 `src/views/Detection.vue`），导入并使用：

```

<template>
  <div>
    <!-- 并排对比模式 -->
    <div v-if="compareMode === 'side'" class="side-by-side">
      <div class="image-card">
        <h4>原图</h4>
        
      </div>
      <div class="image-card">
        <h4>AI标注结果</h4>
        
      </div>
    </div>

    <!-- 滑块对比模式 -->
    <div v-if="compareMode === 'slider'" class="slider-container">
      <SliderCompare
        :before="originalImage"
        :after="annotatedImage"
      />
    </div>
  </div>
</template>

```

```
<script setup>
import { ref } from 'vue'
import SliderCompare from '@/components/SliderCompare.vue'

const compareMode = ref('side') // 'side' 或 'slider'
const originalImage = ref('')
const annotatedImage = ref('')
</script>
```

5.3 Element Plus 自动导入配置

修改 `vite.config.js` 文件:

```
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
import AutoImport from 'unplugin-auto-import/vite'
import Components from 'unplugin-vue-components/vite'
import { ElementPlusResolver } from 'unplugin-vue-components/resolvers'

export default defineConfig({
  plugins: [
    vue(),
    AutoImport({
      resolvers: [ElementPlusResolver()],
    }),
    Components({
      resolvers: [ElementPlusResolver()],
    }),
  ],
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: 'http://localhost:8000',
        changeOrigin: true
      }
    }
  }
})
```

5.4 前端项目启动验证

```
npm run dev
```

访问 `http://localhost:5173`, 看到 Vue 默认页面即为成功。

六、GitHub 代码托管配置

6.1 SSH 密钥配置（免密提交）

1. 生成 SSH 密钥：

```
# 生成ed25519类型的密钥（比rsa更安全）
ssh-keygen -t ed25519 -C "你的GitHub注册邮箱"
```

连续按三次回车，使用默认路径和空密码

2. 将公钥添加到 GitHub

```
# 复制公钥内容到剪贴板（Mac专属命令）
pbcopy < ~/.ssh/id_ed25519.pub
```

然后：

1. 登录 GitHub → 点击右上角头像 → Settings → SSH and GPG keys
2. 点击 "New SSH key"
3. Title 填写 "MacBook Pro 2019"
4. Key 内容粘贴刚才复制的公钥
5. 点击 "Add SSH key"

6.2 项目仓库创建与提交（最终修正版，100% 格式正确）

问题说明：之前的 Windows `echo` 命令会将所有内容输出为一行，导致.gitignore 格式完全错误。强烈推荐
使用手动创建方式，这是最可靠、不会出错的方法。

1. 在 GitHub 创建新仓库，命名为 `rsod-web-platform`，不要勾选"Initialize this repository with a README"
2. 本地项目初始化 Git（推荐：手动创建.gitignore 文件）

```
# 进入项目根目录（包含backend和frontend文件夹的上层目录）
cd rsod-web-platform

# 初始化Git仓库
git init
```

手动创建.gitignore 文件（关键！）

- 在项目根目录右键→新建文本文档→重命名为 `.gitignore`（注意去掉.txt 后缀）
- 用 VS Code 或记事本打开，完整复制粘贴以下内容（格式完全正确）：

```
# =====
# Python 后端相关
# =====
__pycache__/
*.py[cod]
*$py.class
*.so
.Python
```

```
build/
develop-eggs/
dist/
downloads/
eggs/
.eggs/
lib/
lib64/
parts/
sdist/
var/
wheels/
*.egg-info/
.installed.cfg
*.egg
MANIFEST
```

虚拟环境

```
.env
.venv
env/
venv/
ENV/
```

日志文件

```
*.log
logs/
```

=====

AI模型与数据相关（必须过滤！）

=====

模型权重文件

```
*.pt
*.pth
*.onnx
*.engine
*.trt
*.weights
*.ckpt
```

数据集

```
data/
dataset/
datasets/
```

运行结果

```
runs/
static/annotated/
static/videos/
static/original/
quarantine/
```

```
# =====
# Vue 前端相关
# =====
node_modules/
.DS_Store
dist/
dist-ssr/
*.local
.npmrc

# 编辑器临时文件
.vscode/*
!.vscode/extensions.json
!.vscode/settings.json
.idea/
*.iml
*.iws
*.ipr
*.swp
.sublime-*

# 系统文件
Thumbs.db
.DS_Store
```

4. 继续执行 Git 命令：

```
# 添加所有文件到暂存区
git add .

# 首次提交（遵循Git提交规范）
git commit -m "feat: 初始化项目环境与基础结构"

# 关联远程仓库（替换为你的GitHub仓库地址）
git remote add origin git@github.com:你的用户名/rsod-web-platform.git

# 推送到远程main分支
git push -u origin main
```

验证提交：刷新 GitHub 仓库页面，确认所有代码已成功上传，且没有出现模型文件、数据集、**node_modules** 等大文件

七、项目整体验证

7.1 后端main.py

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()
```

```
# 允许 Vue 前端跨域访问
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"], # Vue 默认端口
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/api/test/connect")
async def test_connect():
    return {"code": 200, "message": "前后端连通成功!"}
```

7.2 后端服务启动

```
# 激活虚拟环境
conda activate rsod-web

# 进入后端目录
cd backend

# 启动FastAPI服务
uvicorn main:app --port 8000
```

访问 `http://localhost:8000/docs`，看到 Swagger 接口文档即为成功。

7.3 前后端连通性验证

在 `frontend/src` 里新建文件夹 `views`

创建纯手动实现的测试页(`test_connect.vue`)

`frontend/src/views/test_connect.vue`

```
<template>
  <div class="wrap">
    <h1>🔍 目标检测系统</h1>
    <button @click="testConnect">测试前后端连接</button>
    <div class="result" v-if="res">{{ res }}</div>
  </div>
</template>

<script setup>
import { ref } from "vue";

const res = ref("");

const testConnect = async () => {
  try {
    const response = await fetch("http://localhost:8000/api/connect");
```

```

    const data = await response.json();
    res.value = data.message;
  } catch (e) {
    res.value = "连接后端失败，请检查后端是否启动";
  }
};
</script>

<style scoped>
.wrap {
  max-width: 800px;
  margin: 0 auto;
  padding: 20px;
}

button {
  padding: 10px 20px;
  font-size: 16px;
  margin: 20px 0;
  cursor: pointer;
}

.result {
  font-size: 18px;
  color: green;
  font-weight: bold;
}
</style>

```

7.4 启动前端（最终）

```
npm run dev
```

访问：

<http://localhost:5173/>

- ✅ 能正常打开页面
- ✅ 点击按钮能连接后端
- ✅ 路由正常工作

这就是标准 Vue Router 架构！

7.5 YOLO 推理验证

调用后端 `/api/inference/single` 接口，上传测试图片，成功返回检测结果和标注图 URL 即为成功。

1. 完整后端代码（`main.py`）

```
from fastapi.middleware.cors import CORSMiddleware
```



```

from fastapi import FastAPI, UploadFile, File
from fastapi.staticfiles import StaticFiles
from fastapi.responses import JSONResponse
from ultralytics import YOLO
import os
import uuid
from PIL import Image

app = FastAPI()
# 挂载静态文件
os.makedirs("static", exist_ok=True)
os.makedirs("static/results", exist_ok=True)
app.mount("/static", StaticFiles(directory="static"), name="static")
# 加载 YOLO 模型（替换为你的模型路径）
model = YOLO("yolo11n.pt") # 或你的自定义模型权重

# 允许 Vue 前端跨域访问
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"], # Vue 默认端口
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# 测试连接接口（已验证可用）
@app.get("/api/test/connect")
async def test_connect():
    return {"code": 200, "message": "前后端连通成功!"}

# 核心推理接口
# 修复后的推理接口
@app.post("/api/inference/single")
async def inference_single(file: UploadFile = File(...)):
    try:
        # 1. 保存临时文件
        temp_filename = f"temp_{uuid.uuid4().hex}.jpg"
        temp_path = os.path.join("static", temp_filename)
        with open(temp_path, "wb") as f:
            f.write(await file.read())

        # 2. YOLO 推理，固定保存目录，避免 exp1/exp2
        results = model(
            temp_path,
            save=True,
            project="static/results",
            name="latest", # 固定目录名，每次覆盖
            exist_ok=True
        )

        # 3. 获取标注图片路径（更可靠的方式）
        result_img_path = os.path.join(
            "static/results/latest",

```

```

        os.path.basename(temp_path)
    )

    # 验证文件是否存在
    if not os.path.exists(result_img_path):
        raise Exception(f"结果图片未生成: {result_img_path}")

    # 4. 生成可访问的 URL
    result_img_url = f"http://localhost:8000/{result_img_path}"

    # 5. 解析检测结果
    detections = []
    for box in results[0].boxes:
        detections.append({
            "class": model.names[int(box.cls[0])],
            "confidence": float(box.conf[0]),
            "bbox": box.xyxy[0].tolist()
        })

    # 6. 清理临时文件
    os.remove(temp_path)

    return {
        "code": 200,
        "message": "推理成功",
        "data": {
            "detections": detections,
            "image_url": result_img_url
        }
    }

except Exception as e:
    # 打印完整错误栈, 方便调试
    import traceback
    traceback.print_exc()
    return JSONResponse(
        status_code=500,
        content={"code": 500, "message": f"推理失败: {str(e)}"}
    )

if __name__ == "__main__":
    uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True)

```

2. 核心页面代码 (src/views/Inference.vue)

```

<template>
  <div class="container">
    <h1>🎯 目标检测系统</h1>

    <!-- 上传组件 -->
    <el-upload

```

```

class="upload-area"
action="#"
:auto-upload="false"
:on-change="handleFileChange"
:before-upload="beforeUpload"
>
<el-button type="primary">选择图片上传</el-button>
<template #tip>
  <div class="el-upload__tip">支持 jpg/png 格式, 大小不超过 5MB</div>
</template>
</el-upload>

<!-- 检测按钮 -->
<el-button
  type="success"
  style="margin-top: 20px"
  @click="handleInference"
  :disabled="!selectedFile || loading"
>
  {{ loading ? "检测中..." : "开始检测" }}
</el-button>

<!-- 结果展示 -->
<div v-if="resultImageUrl" class="result-container">
  <h3>检测结果: </h3>
  
  <div class="detections-list">
    <el-tag v-for="(item, index) in detections" :key="index" type="success">
      {{ item.class }}: {{ (item.confidence * 100).toFixed(1) }}%
    </el-tag>
  </div>
</div>
</div>
</div>
</template>

<script setup>
import { ref } from "vue";
import axios from "axios";
import { ElMessage } from "element-plus";

const selectedFile = ref(null);
const loading = ref(false);
const resultImageUrl = ref("");
const detections = ref([]);

// 文件选择
const handleFileChange = (file) => {
  selectedFile.value = file.raw;
  resultImageUrl.value = "";
  detections.value = [];
};

```

```

// 上传前校验
const beforeUpload = (file) => {
  const isImage = ["image/jpeg", "image/png"].includes(file.type);
  const isLt5M = file.size / 1024 / 1024 < 5;
  if (!isImage) {
    ElMessage.error("请上传 jpg/png 格式图片");
    return false;
  }
  if (!isLt5M) {
    ElMessage.error("图片大小不能超过 5MB");
    return false;
  }
  return true;
};

// 调用推理接口
const handleInference = async () => {
  if (!selectedFile.value) return;

  loading.value = true;
  const formData = new FormData();
  formData.append("file", selectedFile.value);

  try {
    const res = await axios.post("http://localhost:8000/api/inference/single", formData, {
      headers: { "Content-Type": "multipart/form-data" },
    });

    if (res.data.code === 200) {
      ElMessage.success("推理成功!");
      resultImageUrl.value = res.data.data.image_url;
      detections.value = res.data.data.detections;
    } else {
      ElMessage.error(res.data.message || "推理失败");
    }
  } catch (err) {
    ElMessage.error(`请求失败: ${err.message}`);
  } finally {
    loading.value = false;
  }
};
</script>

<style scoped>
.container {
  text-align: center;
  padding: 50px;
}
.upload-area {
  margin: 20px 0;
}
.result-container {

```

```

margin-top: 30px;
}
.result-img {
  max-width: 800px;
  border: 1px solid #eee;
  border-radius: 8px;
}
.detections-list {
  margin-top: 15px;
  display: flex;
  gap: 10px;
  flex-wrap: wrap;
  justify-content: center;
}
</style>

```

3. 完整路由配置 (src/router/index.js)

```

// router/index.js
import { createRouter, createWebHistory } from "vue-router";
// import test_connect from "../views/test_connect.vue"; // 前后端连接测试页面
import inference from "../views/Inference.vue"; // Yolo 推理验证页面
// 路由配置
const routes = [
  {
    path: "/",
    name: "inference",
    component: inference, // 默认打开就是检测页面
  },
];

// 创建路由实例
const router = createRouter({
  history: createWebHistory(),
  routes,
});

export default router;

```

4. 完整测试流程

步骤 1: 启动后端服务

```
uvicorn main:app --reload --port 8000
```

步骤 2: 启动前端服务

```

npm run dev
# 服务运行在 http://localhost:5173

```

步骤 3：测试接口连通性

- 访问 `http://localhost:8000/docs`，查看 `/api/test/connect` 接口是否正常
- 前端点击「测试前后端连接」按钮，确认返回成功

步骤 4：YOLO 推理验证

1. 点击「选择图片上传」，上传一张测试图片（如包含人 / 车 / 动物的图片）
2. 点击「开始检测」
3. 成功标志：
 - 前端显示标注后的图片
 - 下方显示检测到的目标类别和置信度
 - 后端控制台无报错

八、常见问题排查

1. CUDA 不可用

- 卸载已安装的 PyTorch: `pip uninstall torch torchvision torchaudio -y`
- 重新安装对应 CUDA 版本的 PyTorch
- 确认 NVIDIA 驱动版本与 CUDA 版本匹配

2. 依赖安装失败

- 使用清华源加速: `pip install -i https://pypi.tuna.tsinghua.edu.cn/simple 包名`
- 升级 pip: `python -m pip install --upgrade pip`
- Windows 用户以管理员身份运行命令提示符

3. 端口占用

- 查看端口占用: `netstat -ano | findstr :8000` (Windows) / `lsof -i :8000` (Linux/macOS)
- 杀死占用进程: `taskkill /F /PID 进程ID` (Windows) / `kill -9 进程ID` (Linux/macOS)

4. 跨域问题

- 后端添加 CORS 中间件:

```
from fastapi.middleware.cors import CORSMiddleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

5. YOLO 模型下载失败

- 手动下载模型权重: [YOLO11 官方权重](#)
- 放入 `backend/models/` 目录，使用本地路径加载: `model = YOLO("models/yolov11n.pt")`

6. **.gitignore** 不生效

- 清除 Git 缓存: `git rm -r --cached .`
- 重新添加文件: `git add .`
- 重新提交: `git commit -m "fix: 更新.gitignore文件"`